KAFKA INGESTION PIPELINE

Study note | Random User API / Airflow / Schema Registry / Kafka | 2 trang

Cốt lõi: API được gọi định kỳ; mỗi user sau khi format và serialize trở thành **một Kafka record**. Airflow chạy một task mô phỏng streaming trong 60 giây.

01 | Luồng xử lý tổng thể

get_data() HTTP GET / list[dict]	format_data() nested / flat dict	stream_data() schema / Kafka Producer

Trong Airflow hiện tại: chỉ có **1 task** tên `stream_data`; hai hàm còn lại là Python functions được gọi trong task này.

02 | get_data(): lấy raw data theo batch

Thao tác	Điểm cần nhớ
`requests.get(API_URL, params={"results": 5}, timeout=30)`	Mỗi request trả về tối đa 5 users theo cấu hình bài học.
`raise_for_status()` + kiểm tra `error`	Phát hiện HTTP/API lỗi thay vì âm thầm xử lý.
`return results`	Kết quả là **list[dict]**, không còn `results[0]` (1 user).

Random User là **HTTP API**, không đẩy dữ liệu liên tục. Vòng lặp gọi API mỗi ~5 giây là **polling mô phỏng streaming**.

03 | format_data(): chuẩn hóa từng user

```
user = format_data(raw_user)

# user_id trong format_data l□y từ res["login"]["uuid"]
# Các trường khác: first_name, last_name, gender, address,
# postcode, email, username, dob, registered_date, phone, picture
```

Đầu ra: một Python dictionary phẳng gồm **12 fields**. UUID là định danh user dùng làm Kafka key; `username` vẫn là một thuộc tính của payload. `postcode` được ép sang string.

04 | Kafka record nhận được gì?

Thành phần	Ví dụ / vai trò
Topic	`randomuser_events_v2`
Key	`user_id` (UUID chuyển thành bytes UTF-8); cùng key thường vào cùng partition với partitioner/số partitions ổn định.
Value	JSON Schema serialized **bytes** (có schema identification + dữ liệu JSON), không phải JSON text thuần.
Partition / Offset	Partition do Kafka Producer chọn theo key; offset do Kafka gán theo từng partition.

05 | Schema Registry trong flow

File `schemas/randomuser.json` định nghĩa 12 fields; `SchemaRegistryClient` kết nối Registry; `JSONSerializer` dùng schema để tạo bytes có Schema ID. Subject mặc định: `randomuser_events_v2-value`.

Nhớ: Một schema dùng cho nhiều records; `auto.register.schemas=True` không có nghĩa tạo schema mới cho mỗi user. Serialization **không phải mã hóa bảo mật**.

STREAM_DATA() · TRỌNG TÂM

Một Airflow task | Khởi tạo Producer một lần; lặp qua các batches; gửi và kiểm tra kết quả

06 | Sáu bước bên trong stream_data()

Bước	Code / kiến thức phải nhớ
1. Chuẩn bị	Đọc env: `KAFKA_TOPIC`, `KAFKA_BOOTSTRAP_SERVERS`, `SCHEMA_REGISTRY_URL`; đọc JSON Schema từ file.
2. Registry + Serializer	`SchemaRegistryClient({"url": ...})` và `JSONSerializer(schema, registry)`. Đăng ký/tra cứu schema khi serialize.
3. Kafka Producer	`Producer({"bootstrap.servers": ..., "acks": "all"})`. 3 địa chỉ broker để bootstrap; `acks=all` kết hợp RF=3, Min ISR=2.
4. Poll API / format	Trong `while` chạy ~60 giây: `get_data()` lấy batch 5 users; `for` duyệt từng user và `format_data()`.
5. Serialize / produce	`value = serializer(user, context)`; `producer.produce(topic, key=UUID bytes, value=value, on_delivery=...)`.
6. Flush / kiểm tra	`flush(30)` chờ delivery; callback thu lỗi; nếu pending hoặc errors còn `raise RuntimeError`.

07 | Lệnh quan trọng: hiểu đúng thứ tự

```
value = serializer(user, SerializationContext(topic, MessageField.VALUE))
producer.produce(topic=topic, key=user["user_id"].encode("utf-8"),
                 value=value, on_delivery=delivery_report)
# ... lặp lại cho các user/batch khác ...
remaining = producer.flush(30)
if remaining or errors:
    raise RuntimeError("Kafka delivery failed")
```

producer.produce() xếp message vào **hàng đợi nội bộ của Producer** để gửi bất đồng bộ - chưa khẳng định Broker đã ghi thành công. **delivery_report(err, msg)** nhận kết quả thành công/thất bại khi client phục vụ callback (qua `poll()` hoặc `flush()`). **flush()** chờ hàng đợi hoàn tất; vẫn phải kiểm tra lỗi callback.

08 | 60 giây chạy thử: đọc log đúng

Thông số	Diễn giải
5 users / request	`BATCH_SIZE=5`; một user tương ứng một Kafka message.
5 giây / lượt	`FETCH_INTERVAL=5`; chờ giữa các HTTP requests.
60 giây / lần chạy	`STREAM_DURATION=60`; là mô phỏng có giới hạn, không phải streaming 24/7.
Log thực tế: 55 users	11 lần "Queued 5 users"; sau `flush()` không có lỗi nên task báo success.

09 | Kiểm thử trước milestone commit

[1] Airflow task success + không có delivery errors. **[2]** Topic: 3 partitions, RF=3, Min ISR=2.
[3] Kiểm tra **55 messages mới** bằng chênh lệch offsets / lọc theo run, tránh nhầm với records cũ.
[4] Control Center: key = UUID, payload đủ 12 fields; Registry: subject, ID, version hợp lệ.

Ba giới hạn cần nhớ: (1) Airflow đang **poll API** theo khoảng thời gian, không phải nguồn push streaming. (2) UUID của Random User thường thuộc user ngẫu nhiên mới mỗi request. (3) Producer idempotence / `acks=all` không tự chống trùng giữa các lần chạy lại DAG.